0435. Non-overlapping Intervals

ium | Category: Intervals | LeetCode: https://leetcode.com/problems/non-overlap

Problem

- Return the minimum number of intervals to remove so the rest do not overlap (touching endpoints are fine).

Intuition

- Same as keeping the maximum number of non-overlapping intervals (activity selection).
- Greedy: sort by end and always keep the interval that finishes first; it leaves the most room for the rest.

Brute Force

- Try all subsets: O(2^n). An LIS-style DP is O(n^2).

Optimal Approach

- Sort by end.
- PrevEnd = -infinity; for each interval: if start >= prevEnd keep it (prevEnd = end); else removed++.

Worked Example

```
[[1,2],[2,3],[3,4],[1,3]] sorted by end: [1,2],[2,3],[1,3],[3,4].

[1,2] keep (end 2)
[2,3] keep (end 3)
[1,3] 1 < 3 -> remove
[3,4] keep
removed 1
```

Java Solution

```java
public class NonOverlappingIntervals {
    public int eraseOverlapIntervals(int[][] intervals) {
        int[][] a = intervals.clone();
        Arrays.sort(a, (x, y) -> Integer.compare(x[1], y[1]));
        int removed = 0;
        long prevEnd = Long.MIN_VALUE;
        for (int[] in : a) {
            if (in[0] >= prevEnd) prevEnd = in[1];
            else removed++;
        }
        return removed;
    }
}
```

Complexity

- Time: O(n log n). Space: O(1) besides sorting.

Pitfalls

- Sorting by start also works if, on overlap, you keep the one with the smaller end; sorting by end is simpler.